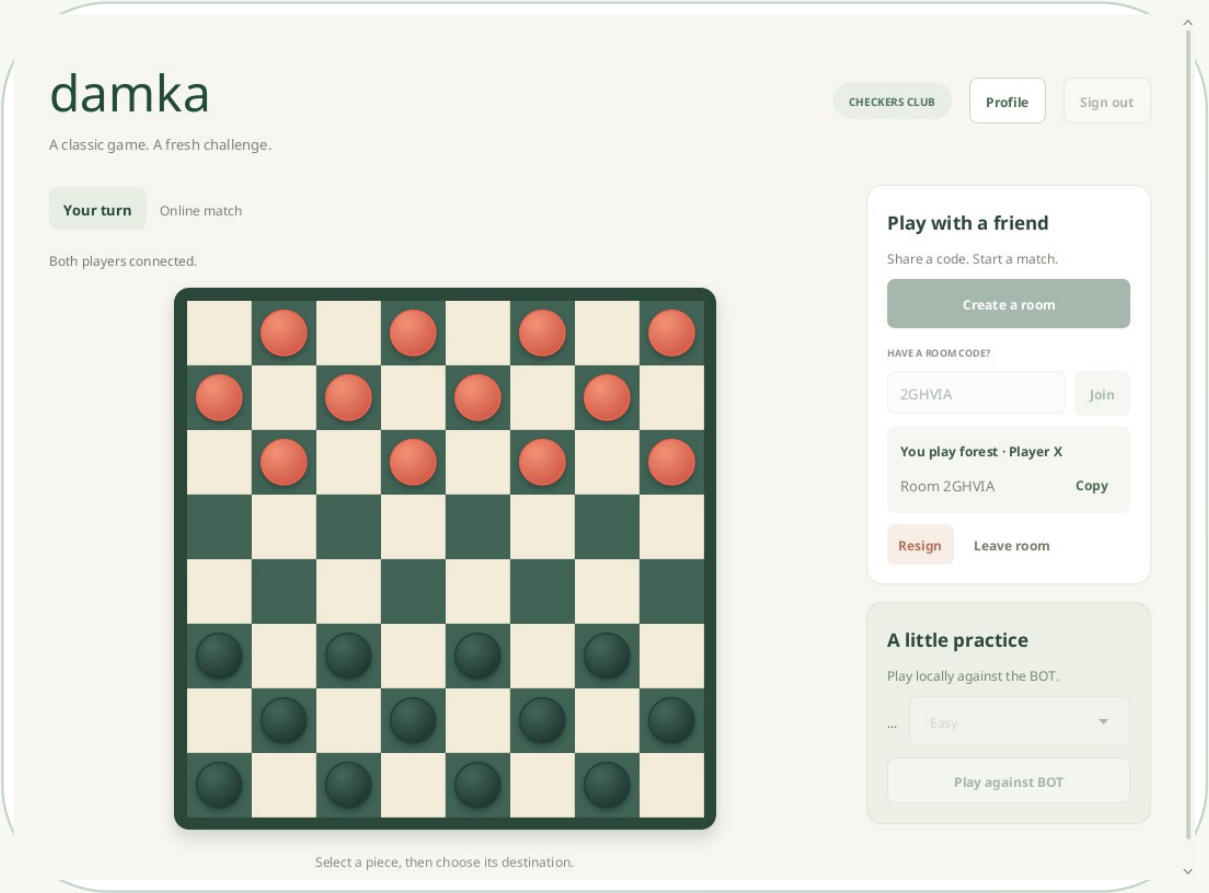
Damka

Checkers, connected.

TCP multiplayer · Local BOT

Accounts, statistics and history



Muthanna Farhat

Rabea Abu Aad

Supervisor: Inbal Ronen

One game, two ways to play

Keep both screens in sync — and remember the result.

Online matches

Two computers
One room code

Local BOT

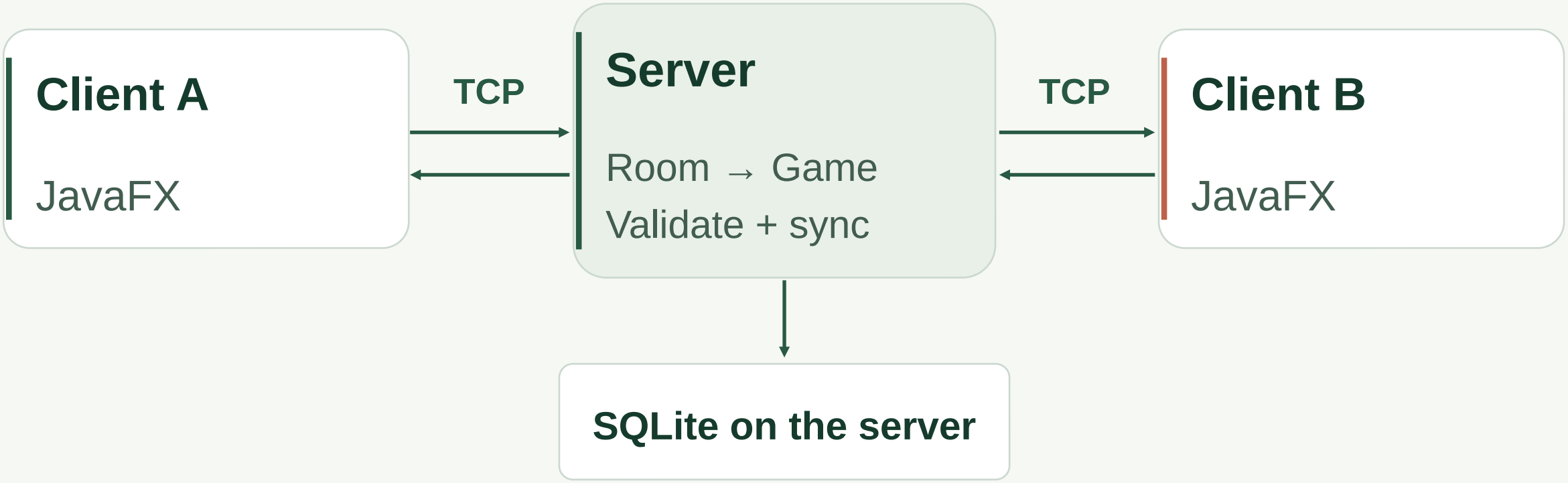
Three difficulty levels
Offline practice

Personal profiles

Statistics
Match history

One shared rules engine: Game

Why a client–server design?



Choice: persistent TCP for two-way updates

Alternative: HTTP polling

From a click to both screens

Client A

Select a piece and destination

```
MOVE 5 0 4 1
```

Server

Room: identity + turn

Game.move(): rules

Accept or reject

Both clients

Accepted move

```
1   MOVE_APPLIED
2   BOARD
3   TURN
```

Animate → display the accepted state

UTF-8 · one message per line · network work off the UI thread

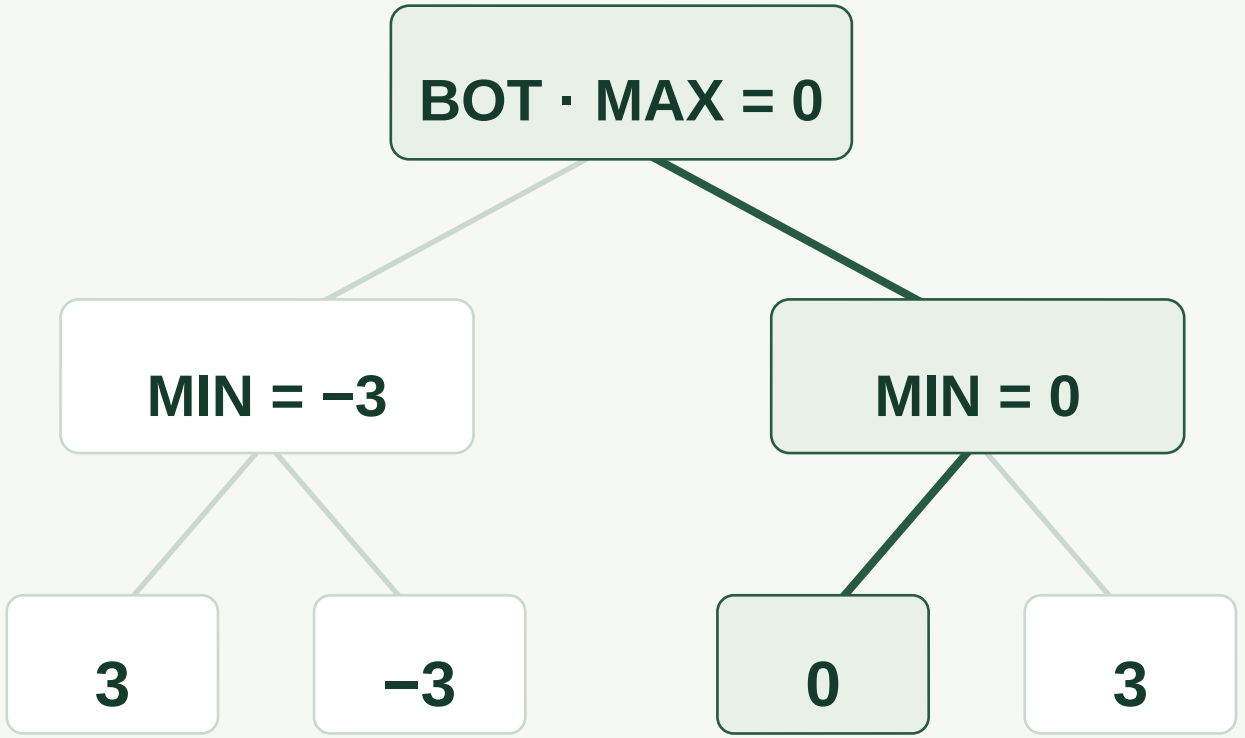
Minimax: how the BOT chooses

Shared game rules

Legal moves on copies

Piece = 3 King = 6

Depth: 1 / 2 / 3



Illustrative two-step search

Alpha–Beta skips branches that cannot improve the decision.

Accounts that remember results

Player profile

Illustrative account

ONLINE

1 win / 1 match

BOT

0 matches

Match history

Online	**WIN**	Resignation

`profiles`

Username · salt · hash

`game_results`

Players · result · mode · time

Saved results → statistics

Verification and AI-assisted development

48 **automated tests**

Game rules and edge cases
Real TCP protocol checks
SQLite persistence and deduplication

Documented passing run: 12 Oct 2026

Codex + GitHub Copilot

Planning · code · explanations
Tests and documentation

Read → run → review → refine

What we built and learned

One authoritative game.
Two synchronized clients.
Persistent player results.

Next: chat

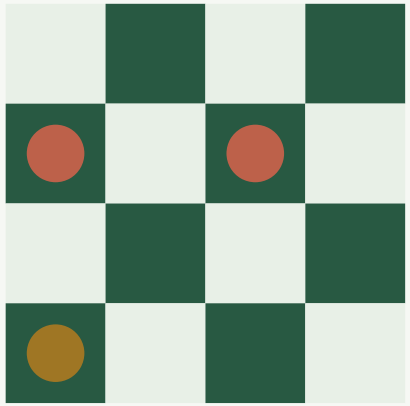
Inside the room
Over existing TCP

Rabea Abu Aad — Game Logic
Muthanna Farhat — TCP Communication

github.com/MuthannaFrhat/damka-game

THANK YOU

Questions & discussion



github.com/MuthannaFrhat/damka-game